

# Midterm Reflection Journal: Full-Stack Web Development

DOCUMENT 4 OF 4 • BACKEND & FULL-STACK RETROSPECTIVE • 25 POINTS RUBRIC

**Student:** Course Participant

**Course Code:**  
6APSI

**Project:** Doton City Pop Vault

**Date:** September 2026

## 1. Frontend Architecture & Prelim Continuity

***Note:** As I previously submitted a Prelim Reflection Journal covering early JavaScript and React UI foundations, this journal focuses predominantly on backend architecture, Node/Express REST APIs, and PostgreSQL/Supabase integration.*

Building the **Doton City Pop Vault** platform required translating core JavaScript concepts and React component architecture into a production-grade full-stack web application. Establishing a modular design system (DOTON\_DESIGN\_SYSTEM.md) using plain CSS custom properties enabled seamless visual consistency and multi-theme switching (dark, white, sunset) across all viewports. The primary frontend challenge involved managing synchronized state across multi-criteria filter bars while maintaining persistent HTML5 audio preview playback across modal dialogs without triggering unwanted component re-renders.

## 2. Backend Architecture: Node, Express, & REST APIs

Transitioning to backend development introduced fundamental concepts in server lifecycle management, HTTP protocol specifications, and RESTful API contracts. My most instructive technical hurdle occurred when integrating the React recommendation submission form with our Node/Express REST backend server.

### Specific Bug Analysis: Postman vs. React Form Submission

- Missing Headers:** The React `fetch()` request omitted 'Content-Type': 'application/json', causing the payload to transmit as an unparsed body stream.
- Middleware Ordering:** The Express application lacked `app.use(express.json())` prior to mounting API routes, leaving `req.body` undefined.
- CORS Policy Preflight:** Fetch requests from Vite (localhost:5173) to Express (localhost:3000) were blocked until mounting the cors middleware.

Resolving these failures reinforced the absolute necessity of explicit request header verification, middleware pipeline ordering, and cross-origin resource sharing (CORS) setup when coupling decoupled client and server environments.

## 3. Database Design & Supabase Troubleshooting: Manual Workflows

Transitioning to PostgreSQL transformed my understanding of relational data modeling, schema integrity, and database security. In `schema.sql`, I architected a relational database schema comprising albums, tracks, and recommendations tables.

### Supabase API Integration Challenge & Manual Resolution:

I encountered integration obstacles connecting automated REST API endpoints directly with Supabase's live client environment due to asynchronous initialization timeouts and environment variable configuration mismatches (`VITE_SUPABASE_URL`). To overcome this while retaining Supabase PostgreSQL benefits, I adopted a hands-on manual workflow:

- Manual Schema & Data Seeding:** Manually executed SQL schema migrations inside Supabase's SQL editor (establishing `tracks.album_id` foreign key cascades) and populated seed datasets for 15+ City Pop albums directly in Supabase's table editor.
- Hybrid Query Layer:** Wrote explicit Supabase JS query calls paired with local state fallbacks to guarantee zero app downtime during API disruptions.

Additionally, handling PostgreSQL data types revealed key constraints: inserting genre strings ("Beach Sunset, City Pop") failed against strict genre `TEXT[]` array columns until formatted as native arrays (`ARRAY['Beach Sunset', 'City Pop']`). Initial public inserts triggered PostgreSQL error 42501: new row violates row-level security policy, which I resolved by defining explicit Row Level Security policies (`CREATE POLICY "Allow public insert" ON public.recommendations FOR INSERT WITH CHECK (true);`).

## 4. Honest AI Utilization & Code Craftsmanship Breakdown

Rather than relying on unguided 'vibe coding' where AI generates unchecked output, I used AI strictly as a pair-programming multiplier. My codebase reflects a **70% Manual Craftsmanship / 30% AI Assistance** distribution:

**Engineering Distribution & Craftsmanship Audit:**

- **Manual Engineering (70%):** Hand-crafted the modular CSS design system (DOTON\_DESIGN\_SYSTEM.md), React audio player context, multi-criteria filter algorithms, manual Supabase SQL schema migrations, RLS security policies, Express middleware stack, and cross-origin fetch headers.
- **AI Pair-Programming (30%):** Accelerated generation of repetitive CSS glassmorphism custom property templates, drafted raw SQL seed scripts for 15+ classic City Pop albums with iTunes preview links, and parsed complex async useEffect stack traces.

AI assistance had clear pitfalls. AI initially recommended an overly complex Redux state tree for managing floating audio playback; refactoring to a lightweight React Context provider proved far more maintainable. AI also suggested raw SQL query strings that bypassed Supabase's JS method chaining. Resolving CORS preflight errors, PostgreSQL array constraints, and manual Supabase data workflows required hands-on, methodical debugging.

---

## 5. Live Demonstration & Video Submission Package

The project walk-through and live feature demonstration (covering audio preview playback, filter bar operations, dynamic theme switching, manual Supabase table verification, and recommendation submission into PostgreSQL) is documented in the video submission link included with this project submission package.